COREPROCESS AND HOW IT COMMUNICATES

The simulator that receives instructions from the UI and carries out the requested work. It runs on its own Java thread. This keeps the command-handling work separate from the code that draws the window and responds to button clicks.

You can think of it as the simulator’s coordinator. The UI asks it to do something, CoreProcess checks the command, calls the required CPU operation, and sends a result back to the UI.

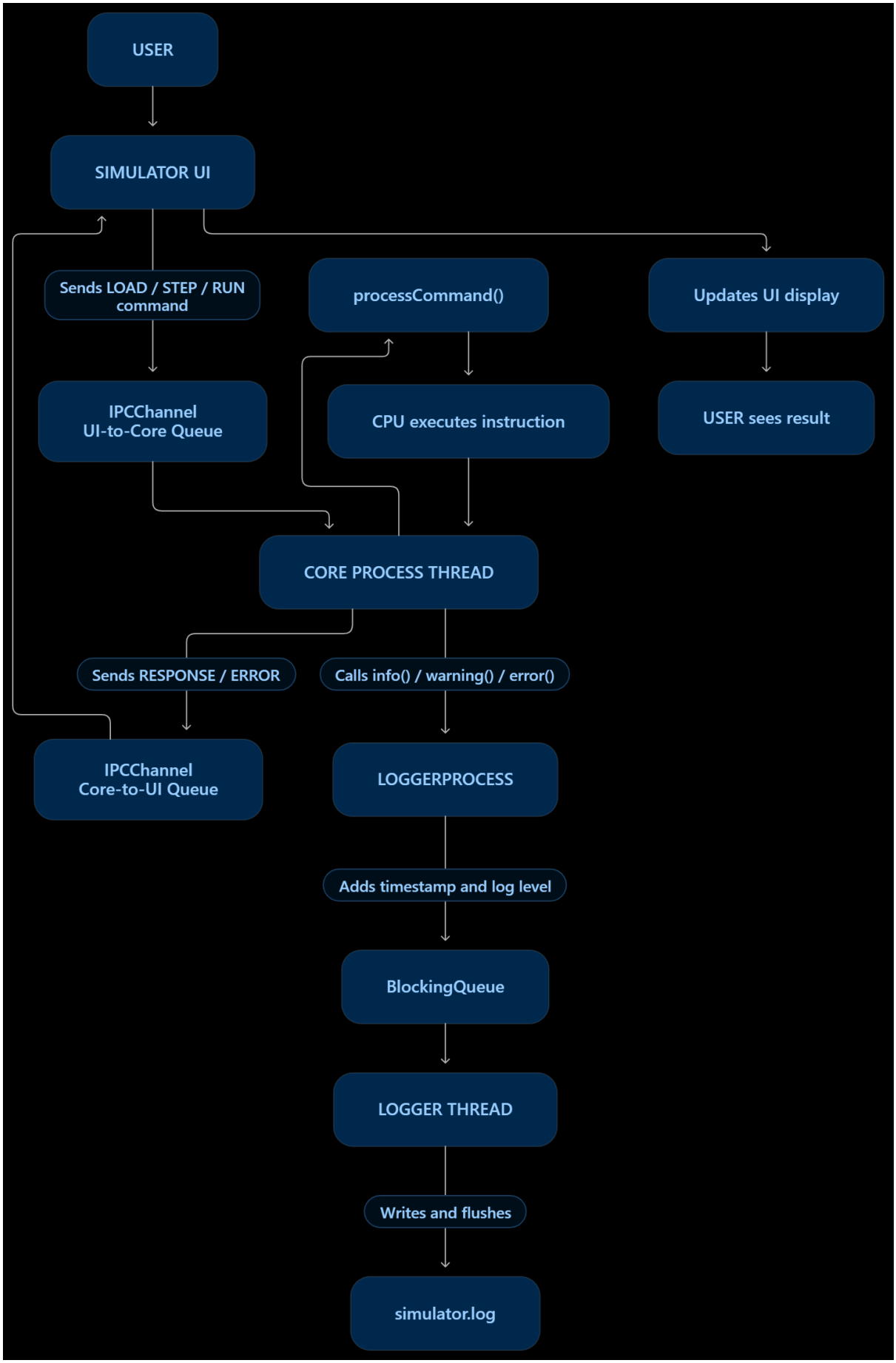
It waits for commands from the UI.

It decides which operation should run, such as LOAD, RESET, STEP or RUN.

It works with the shared CPU object, so the UI and CoreProcess see the same registers, memory, flags and queue state.

It sends a RESPONSE when an operation succeeds, or an ERROR message when an operation fails.

CoreProcess handles simulator commands. LoggerProcess records important events in a log file. CoreProcess passes log messages to LoggerProcess, and LoggerProcess writes them on its own thread.


USER
SIMULATOR UI
Sends LOAD / STEP / RUN command
processCommand()
Updates UI display
IPCChannel UI-to-Core Queue
CPU executes instruction
USER sees result
CORE PROCESS THREAD
Sends RESPONSE / ERROR
Calls info() / warning() / error()
IPCChannel Core-to-UI Queue
LOGGERPROCESS
Adds timestamp and log level
BlockingQueue
LOGGER THREAD
Writes and flushes
simulator.log